

1. 实验目标 (0.5页)

- 明确任务：通过源码分析数据库如何解析并执行SQL语句（如 `SELECT/UPDATE/JOIN`）
- 简单对比目标系统与其他数据库（如MySQL/PostgreSQL/SQLite）的架构差异

2. 系统架构总览 (1.5页)

- 核心模块图示**（手绘或工具生成架构图，标注解析器、优化器、执行引擎、存储引擎）
- 关键代码目录**（示例：MySQL的 `sql/` 目录存放解析器和执行器代码）
- 优势设计点**（如SQLite的单一文件存储 vs MySQL的插件式存储引擎）

3. SQL语句执行过程拆解 (5页, 重点部分)

选择3-4类典型语句（如SELECT查询、带索引的UPDATE、多表JOIN），每类1-1.5页：

- 代码跟踪路径**（示例：`SELECT * FROM table` 的调用链：`mysql_parse()` → `handle_query()` → `execute_sqlcom_select()`）
- 关键数据结构**（如 `TABLE` 对象、`THD` 线程描述符）
- 执行阶段可视化**（流程图说明：解析→优化→执行）
- 对比分析**（例：有无索引时 `WHERE` 条件处理的代码分支差异）

4. 查询流程优势与局限性 (1页)

- 优势分析**（如：
 - ✅ 解析器与执行器解耦，便于扩展新语法（代码示例：`sql_yacc.yy` 语法规则文件）
 - ✅ 基于代价的优化器动态选择执行路径（对比 `EXPLAIN` 输出与 `optimizer_trace` 代码）
- 局限性发现**（如：
 - ⚠️ 单线程查询处理导致复杂查询阻塞（定位到 `pthread_mutex_lock()` 调用点）
 - ⚠️ 内存分配策略简单导致大表扫描性能下降（分析 `alloc_root()` 函数调用频率）

5. 总结 (0.5页)

- 用1-2个具体代码片段说明"数据库如何处理一条SQL"
- 总结该架构设计对开发者/使用者的启发